Vashasetu (ভাষাসেতু)

A Regional Bangla Dialect Translator and Learning App

Course Project Report

CSE 416 – Software Project Design and Development

Department of CSE | Autumn 2026

Submitted by:

Name	ID	Section
Shobnom Sultana Muskan	0432320005101142	7B1
Sadia Akter Liza	0432320005101110	7B1

Submitted To:

Dr. Mahfida Amjad Dipa

Assistant Professor, CSE Dept.

Date: 09.11.2026

Declaration:

This project has not been submitted to any other course projects. The use of any assistive tool in this work has been disclosed in the Tool Usage Log (Appendix A).

1. Introduction

1.1 Introduction

Bangladesh has one written standard, Standard Bangla (Cholito Bhasha), but people in different regions speak strongly different regional dialects. Chattogram (Chittagonian), Sylheti and Noakhali speech can be hard to follow for other Bangla speakers, and small indigenous communities (Chakma, Marma, Garo and others) often do not know Standard Bangla, or even its letters, well. This creates real problems in hospitals, offices, schools and markets, and for people who move between regions for work or study.

Vashasetu (ভাষাসেতু, "language bridge") is a progressive web application (PWA) that (i) translates between Standard Bangla, English and eleven regional Bangla dialects, between two dialects through Standard Bangla, and to and from Garo with a word list reviewed by native speakers; (ii) explains every translated word in a word box and shows the word pairs on a Bridge screen; (iii) offers a browsable and editable dictionary for each language; and (iv) teaches through a learning path that starts from the Bangla script, with the instructions shown in the learner's own language. Input can be typed in Bangla script or Roman letters (Banglish), spoken, or scanned from a photo. The app runs on PHP 8 and MySQL (XAMPP), works in any modern browser and can be installed on a phone like a native app.

The target users are: people travelling or working in another region; students and teachers; health and service workers who meet patients and customers from other districts; members of indigenous communities learning Bangla; and native speakers who want to record and correct the words of their own dialect.

1.2 Motivation

Dialect differences are a daily source of misunderstanding, and dialect speakers are sometimes laughed at for their speech. General translators such as Google Translate support Standard Bangla only, so a sentence such as "আইঁ ভাত খাইতান ন" (Noakhali, "I will not eat rice") is not understood. Several research groups in Bangladesh have published dialect corpora (Vashantor, ONUBAD, Kothon, Dwadash, BD-Dialect, MELD) [4]–[10], but these datasets are spreadsheets for machine-learning research and are not usable by ordinary people.

A software solution is needed because the knowledge is large (tens of thousands of sentence pairs), changes as speakers correct it, and must be available on the phone where the conversation happens. A web app can combine the research data, rules confirmed by native speakers and a learning module in one place, at no cost to the user.

In our CSE 355 system model we analysed the root causes of the problem with a fishbone (Ishikawa) diagram (Fig. 1.1): technology, people, process and society all contribute to dialect speakers being left out of digital services.

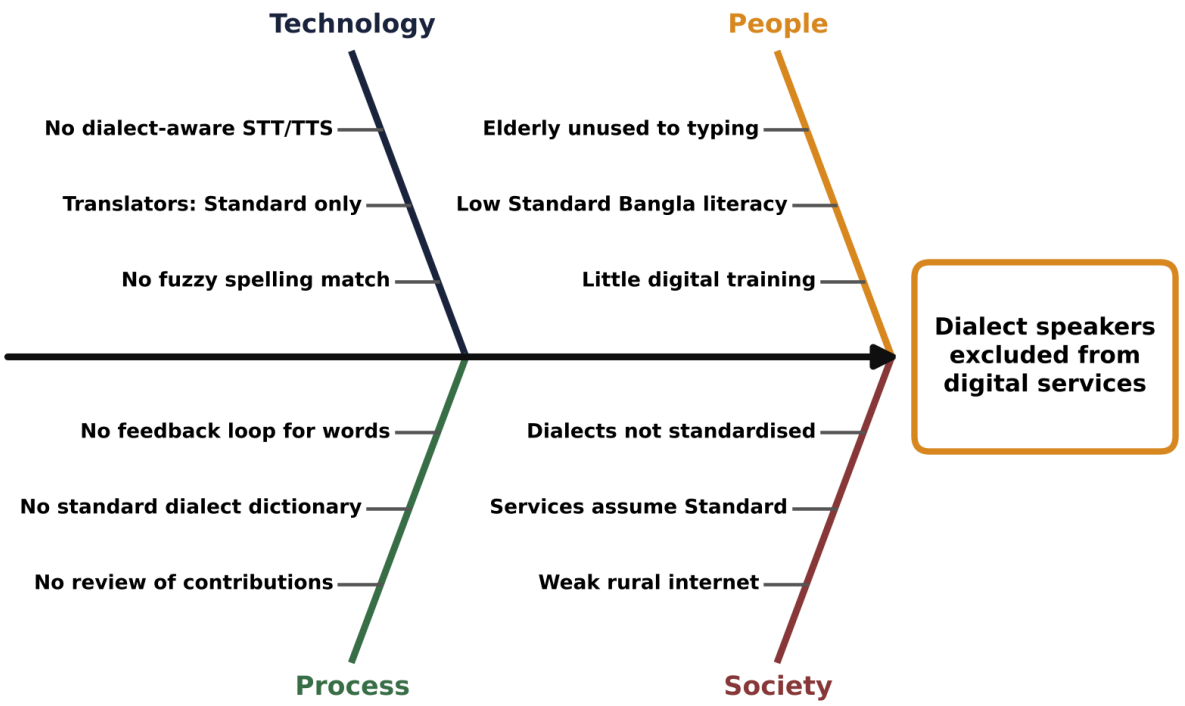


Fig. 1.1 Fishbone diagram – root causes of dialect speakers being excluded from digital services

1.3 Aims and Objectives

Aim: to build and deploy a free, installable web application that helps people understand, translate and learn regional Bangla dialects, with data that native speakers can check and correct.

Objectives (each is measurable and is checked in Section 5):

- O1 – Translate in both directions between Standard Bangla and at least six regional dialects, and between two dialects through Standard Bangla.
- O2 – Accept Bangla script, Banglish (Roman letters), English, voice and photo input.
- O3 – Show the meaning of every word of a translation (word boxes) and mark how reliable each one is.
- O4 – Provide a dictionary for every language, editable by users, with every edit logged and reversible.
- O5 – Provide a learning path (sounds, letters, phonetics, words, sentences) with saved progress for logged-in users.
- O6 – Reach at least 75% word-level accuracy (dialect → Standard) on held-out corpus sentences for the six main dialects.
- O7 – Deploy the app over HTTPS so that it can be opened and installed on a phone, with basic security hardening.

1.4 Scope and Limitations

In scope: translation for Standard Bangla, English, six main dialects (Barishal, Chattogram, Mymensingh, Noakhali, Rajshahi, Sylheti), five further Dwadash dialects with less data (Rangpur, Narail, Kishoreganj, Narsingdi, Tangail), exact-sentence translation for Chakma, Marma and Garo (MELD data), and word-level Garo translation with its native-speaker reviewed word list; word boxes and the Bridge screen; dictionary with editing; learning path and quizzes with instructions in the learner's language; history, streak and word of the day; a district map; user accounts; PWA install.

Out of scope: neural machine translation, audio recordings of dialect speech, and full grammars of the indigenous languages (the indigenous languages are covered only by the sample phrases in MELD).

Assumptions and limitations: the Noakhali, Chattogram, Barishal and Garo word lists were reviewed by native speakers in November 2026; for the other dialects most word pairs are learned automatically from parallel corpora and are labelled "needs review", and only curated entries and native-speaker confirmations are treated as trusted. For indigenous languages only reviewed words are used, so a word missing from the Garo list is left as it is instead of being guessed. Learning instructions are in Standard Bangla for Bangla varieties; for Mro and other indigenous languages without reviewed text the app shows pictures and symbols until a native speaker fills in database/learn_ui.csv. Dialect spelling is not standardised, so the same word can appear in several spellings. Voice input depends on the browser's speech service and Listen needs a Bangla voice on the device. Deployment uses a free Cloudflare quick tunnel from a laptop, so the public link changes when the tunnel is restarted.

1.5 Team Organization and Individual Contributions

The team has two members. Responsibilities follow the work breakdown structure (WBS) of our project proposal: requirements and design were shared. The proposal planned the screens and API integration (front end) for Sadia Akter Liza, but in practice Shobnom Sultana Muskan built both the back end (translation engine, dictionary, voice) and the front end (screens, navigation, API integration), and wrote the automated tests. Sadia Akter Liza did the manual testing of the app on phones. Table 1.1 shows the actual division.

Table 1.1. Division of work (proposal WBS and actual owner)

WBS	Work package	Owner
1.0	Requirements analysis and SRS	Both
2.0	System design (architecture, UML, ER, UI)	Both
3.0	Dictionary module and translation engine (back end, data)	Muskan
4.0	Voice module (speech input, Listen, OCR)	Muskan
5.0	Development of screens and API integration (front end)	Muskan
6.0	Testing and deployment	Muskan (automated tests, deployment); Liza (manual testing)

Work was integrated through one GitHub repository (https://github.com/Muskan37/Vashasetu). Native speakers reviewed the word lists in shared Google Sheets, and their corrections were loaded into the app (Section 4.3). Progress was tracked with a task checklist and the README change log. Individual contributions and the commit record are given in Appendix B.

2. Background Study and Requirement Analysis

2.1 Introduction

This section reviews existing solutions and the research datasets we used, identifies the stakeholders, summarises the Software Requirements Specification (SRS) and compares the alternative solutions we considered. Requirements were gathered from our CSE 355 proposal and system model, conversations with dialect speakers, and by testing what existing tools do with dialect sentences.

2.2 Related Work and Existing Solutions

Google Translate and similar tools translate Standard Bangla well but treat dialect sentences as Standard Bangla, so dialect words are left unchanged or mistranslated. In research, Vashantor [4], ONUBAD [5], Kothon [6], Dwadash [7], ChatgaiyyaAlap [8] and BD-Dialect [9] provide parallel corpora between dialects and Standard Bangla, and MELD [10] provides Chakma, Marma and Garo phrases. Papers built on them train sequence-to-sequence models, but the models are not available as tools for ordinary users and do not explain individual words. Table 2.1 compares these with Vashasetu.

Table 2.1. Comparison of existing solutions with the proposed system

Feature	Google Translate	Research dialect corpora / models [4]–[10]	Vashasetu (proposed)
Standard Bangla ↔ English	Yes	No	Yes (dictionary-based)
Regional dialects	No	Yes, as datasets / trained models	Yes, 11 dialects + 3 indigenous (phrases)
Dialect ↔ dialect	No	No	Yes, via Standard Bangla
Explains each word	No	No	Yes, word boxes with reliability
Users can correct data	Suggest only	No	Yes, edit log with undo
Native-speaker rules	No	No	Yes
Learning module	No	No	Yes, script → words → sentences
Banglish / voice / photo input	Partly	No	Yes
Cost / availability	Free, online	Research use	Free, installable PWA

2.3 Stakeholders and User Characteristics

Stakeholders were identified from the proposal and refined during development:

- **Dialect learners and travellers** – need quick, simple translation on the phone, often by voice, and want to know what each word means.

- **Indigenous-language speakers** (Chakma, Marma, Garo) – may not read Bangla letters, so they need a path that starts with the script.
- **Native speakers / contributors** – know the right dialect word and want to correct the dictionary without technical steps.
- **Service workers** (health, offices, schools) – need reliable phrases and a clear warning when the app is unsure.
- **Administrators (the team)** – load and check data, review and undo edits, deploy and keep the system secure.
- **Dataset authors** – require correct credit and licence compliance (CC BY 4.0, MIT).

Users have mixed technical skills, use mostly low-cost Android phones and may type Bangla in Roman letters, so the interface uses large buttons, Bangla and English labels, and accepts Banglish.

2.4 Software Requirements Specification

The SRS was written from the proposal and updated as requirements changed (for example, after native speakers corrected Noakhali words). Its main points are:

Table 2.2. Summary of the Software Requirements Specification

ID	Requirement	Type
FR1	Translate Standard ↔ dialect, dialect ↔ dialect (via Standard), and to/from English	Functional
FR2	Accept Bangla, Banglish, English, voice (with Stop) and photo (OCR) input; auto-detect the input variety	Functional
FR3	Use an exact sentence translation when a trusted one exists; otherwise word-by-word only if the user allows it	Functional
FR4	Show a word box with the meaning and reliability of every word	Functional
FR5	Dictionary per language: search, browse, add, edit (dialect word and meaning) and delete; verified words first	Functional
FR6	Log every dictionary edit with time and IP address; let the administrator undo all edits from one address	Functional
FR7	Learning path and quizzes per language pair; script lessons first for learners of a Bangla variety; instructions in the learner's own language (never English for indigenous learners)	Functional
FR8	Accounts (register, login), history, streak, word of the day, district map	Functional
NFR1	Usability: works at 360 px phone width, Bangla and English labels, installable PWA, works offline for the app shell	Non-functional
NFR2	Performance: a translation returns in under 1 s on a laptop server; under stress (100 users sending requests non-stop) 95% of replies within 3 s	Non-functional
NFR3	Security: hashed passwords, prepared SQL statements, login throttling, secure cookies, protected data folders	Non-functional
NFR4	Honesty: unverified meanings are labelled; dataset sources and licences are credited	Non-functional
C1	Constraints: zero budget, PHP 8 + MySQL on XAMPP, free hosting, two developers, one semester	Constraint

2.5 Alternative Solutions and Feasibility

We considered three alternatives:

- **A1 – Neural machine translation model** (fine-tuning a model such as BanglaT5 on the corpora). Technically strongest for fluent sentences, but it needs a GPU to train and to serve, cannot explain words, and is hard to correct when a native speaker reports an error. Economically not feasible for us.
- **A2 – Native mobile app** (Android). Good offline use, but it needs separate builds and store publishing, and only one of us has Android experience.
- **A3 – Web app (PWA) with a data-driven engine** (chosen). PHP 8 + MySQL run free on XAMPP, the app installs from the browser, and the engine combines exact sentence matches, a word lexicon learned from the corpora (IBM Model 1 alignment [11]), curated entries and native-speaker rules. Every result can be traced to its source and corrected immediately.

Feasibility: technically A3 uses technologies taught in our courses; economically it costs nothing (open data, free tunnel); operationally it can be used on any phone with a browser and maintained by editing CSV files and the dictionary. Its weakness, rougher word-by-word output, is handled by preferring exact matches and clearly labelling approximate results.

3. System Design and Methodology

3.1 Introduction

This section describes how the project was planned, the architecture and technology stack, the system models, the user interface and the main design decisions. The design follows a layered, data-driven approach: research data and native-speaker knowledge are kept in versioned CSV files, loaded into MySQL, and used by a rule-and-statistics translation engine behind a JSON API.

3.2 Development Methodology and Project Plan

We used an iterative, incremental methodology: requirements and design first (carried over from CSE 355), then short build–test–review cycles in which each version was tried by us and by native speakers, and their corrections became requirements for the next cycle. Fig. 3.1 shows the planned 14-week schedule. Issues were tracked as a checklist in our shared notes and in the README change log; milestones were the mid-term demo, the security review and the final release.

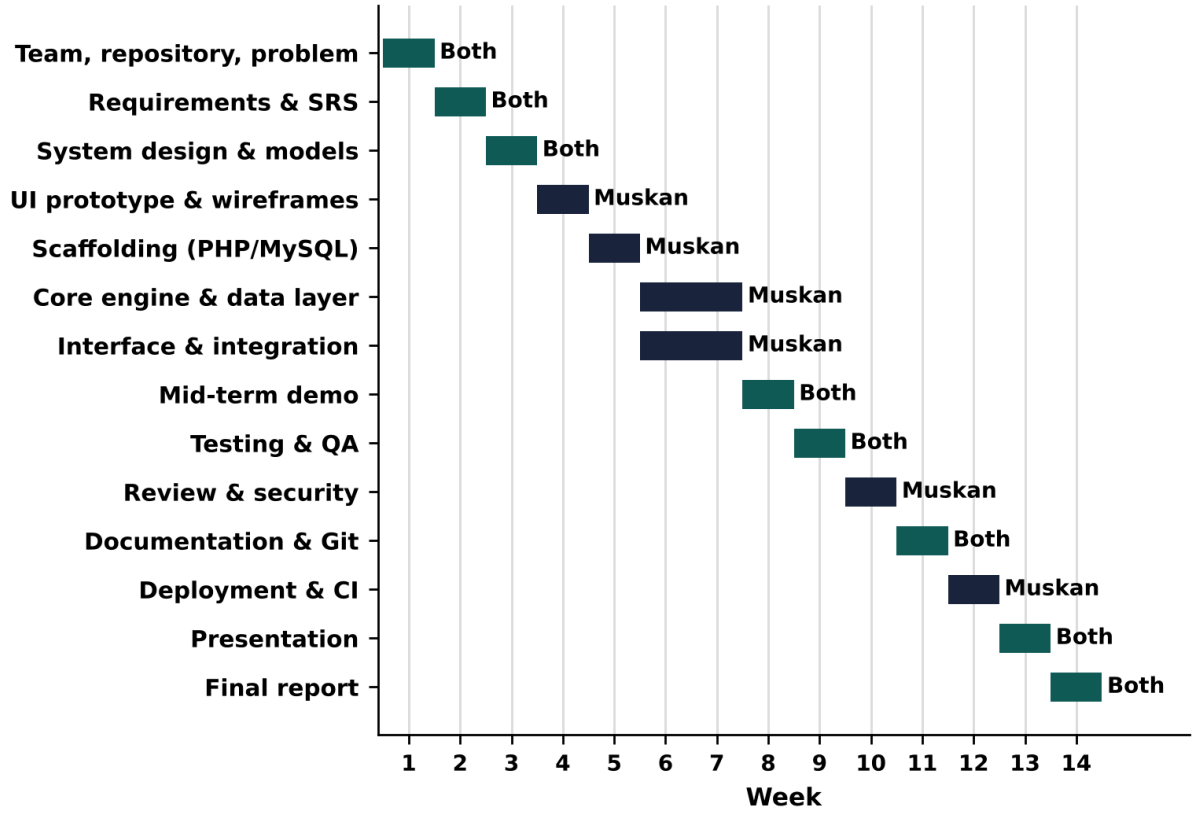


Fig. 3.1 Planned 14-week project schedule (Gantt chart)

In practice most implementation commits were made in the last weeks (19 October – 3 November 2026), because the first versions were built and tested locally on XAMPP and uploaded to GitHub only once they worked (see Section 4.2). Week 13 was used for the full testing campaign and week 14 for native-speaker data, the learner-language Learn page and the final fixes.

3.3 System Architecture and Technology Stack

The system models below were first drawn in our CSE 355 system model (component, activity and sequence diagrams) and have been redrawn to match the final build: speech now uses the browser's Web Speech API instead of Whisper / Google Cloud TTS, and admin approval of contributions became a logged, reversible edit system. Fig. 3.2 shows the component diagram (architecture). The browser client (HTML, CSS, JavaScript, service worker) calls PHP endpoints in api/ that return JSON. The endpoints use a shared text library (normalisation, tokenisation), the matching engine and the data in MySQL. A setup script loads the CSV data folder into the database.

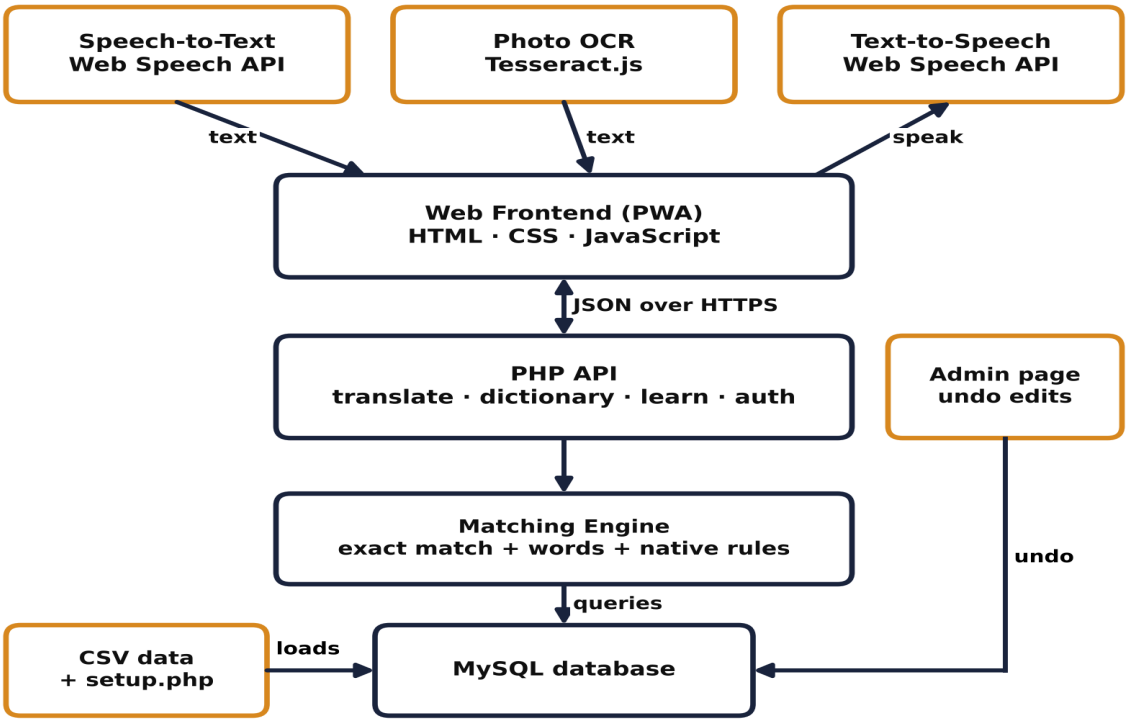


Fig. 3.2 Component diagram (system architecture) of Vashasetu

Table 3.1. Technology stack and reason for each choice

Layer	Technology	Reason
Client	HTML5, CSS3, vanilla JavaScript; PWA manifest and service worker	No build step; installable; offline app shell
Input	Web Speech API (voice), Tesseract.js (OCR, Bengali model)	Free, runs in the browser
Server	PHP 8 (XAMPP / Apache)	Taught in our courses; free; easy to host
Database	MySQL 8 / MariaDB	Relational data, prepared statements
Data tools	Python 3 (csv, openpyxl) scripts in tools/	Checking and importing datasets
Version control / CI	Git, GitHub, GitHub Actions	History, collaboration, automated tests
Deployment	Cloudflare quick tunnel (HTTPS) from the XAMPP laptop	Free public HTTPS link for phones
Design / modelling	Graphviz, Matplotlib, draw.io, Figma (proposal stage)	UML and ER diagrams, wireframes

3.4 System Models

The use-case diagram (Fig. 3.3) shows the actors: a guest can translate, browse the dictionary and edit words; a registered learner also saves progress and history; the administrator loads data and reviews or undoes edits.

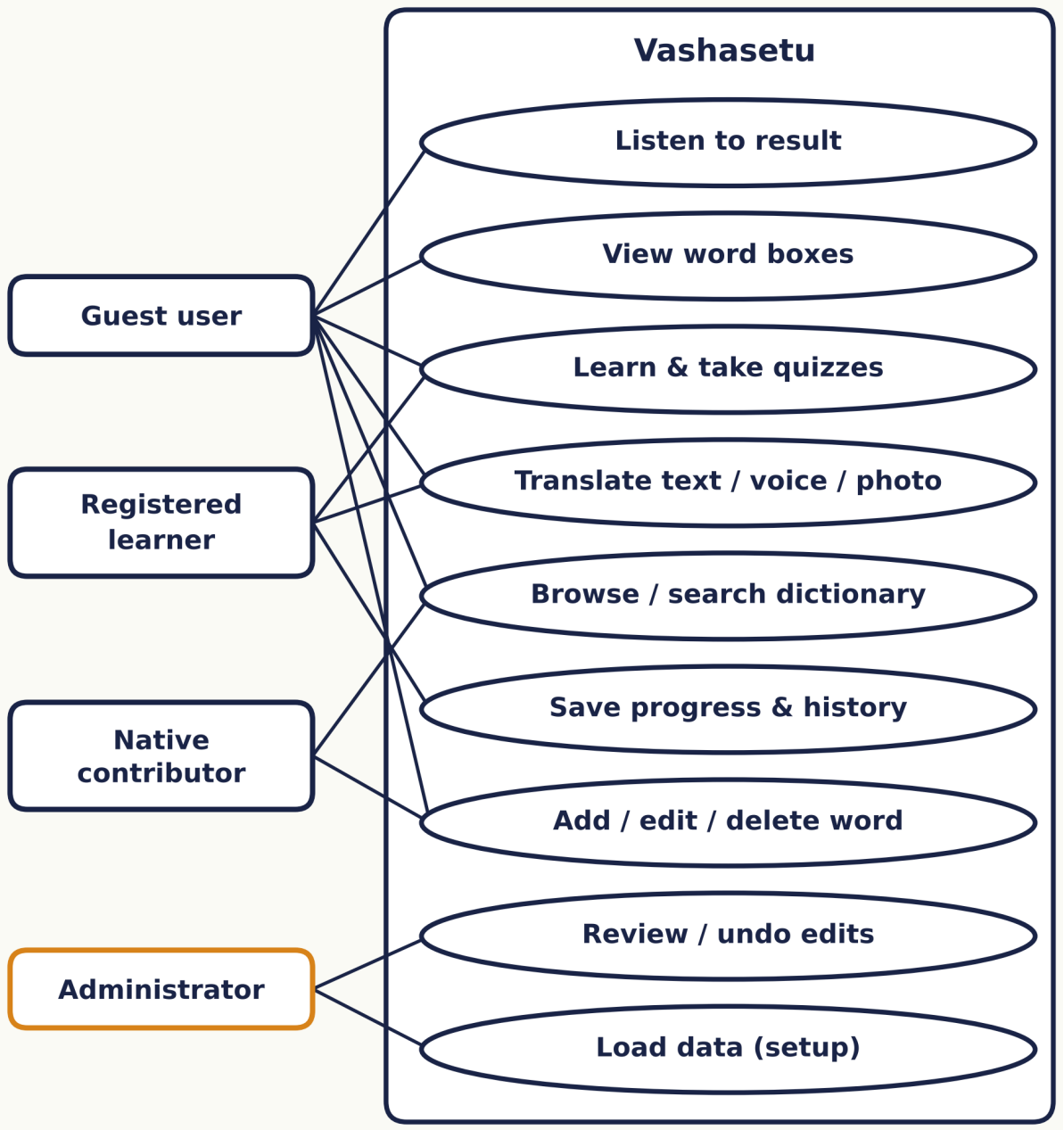


Fig. 3.3 Use-case diagram

The ER diagram (Fig. 3.4) shows the main tables. Each language has its own sentence and word data; word_meanings stores the dictionary with a status (native_speaker_reported, community, corpus); lexicon holds the learned word pairs; users own translation_history, learning_progress and contributions; login_attempts supports throttling. Dictionary edits are also logged per IP address in CSV files so they can be undone.

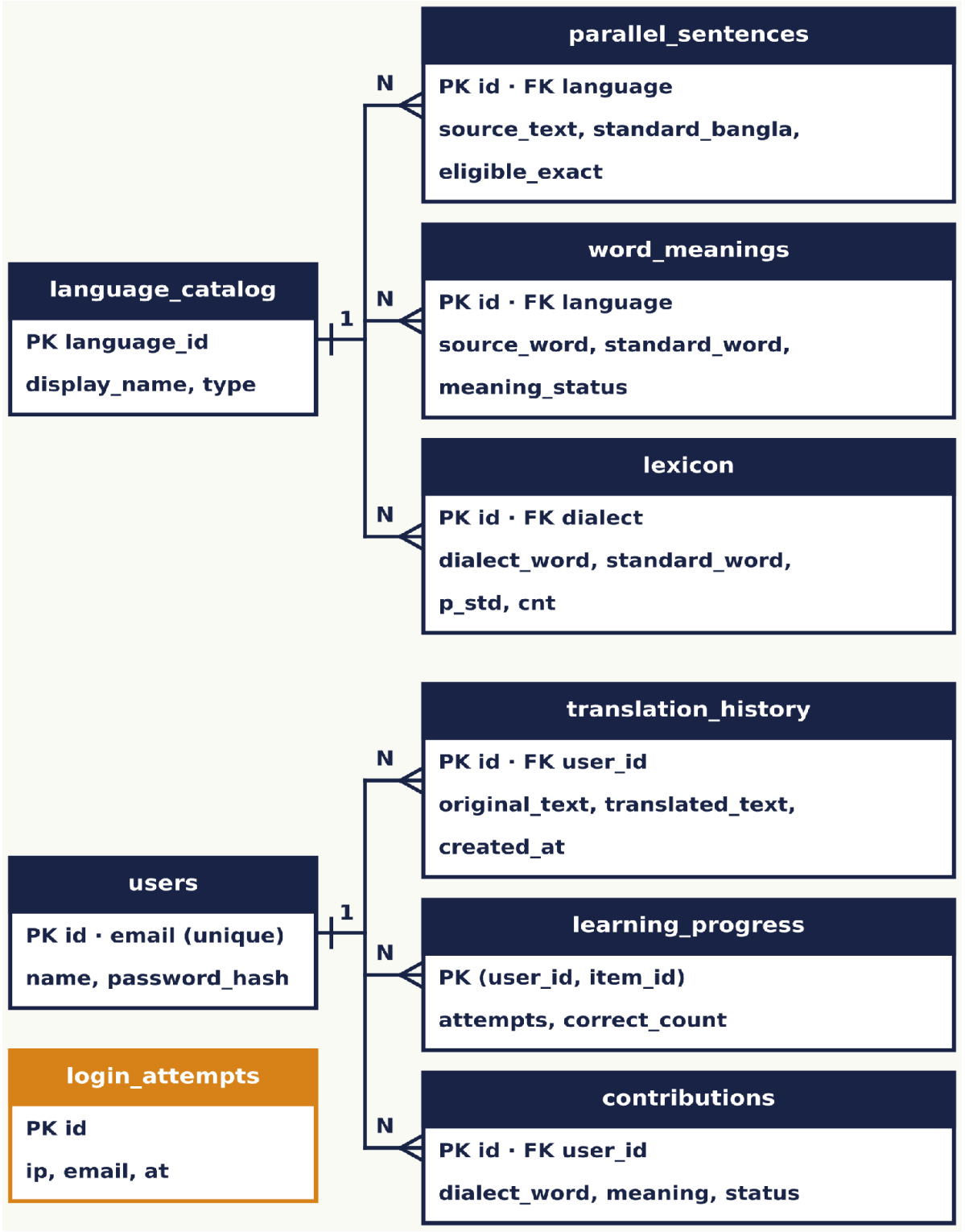


Fig. 3.4 Entity–relationship diagram

Fig. 3.5 shows the main modules (PHP files act as classes with clear responsibilities) and their dependencies.

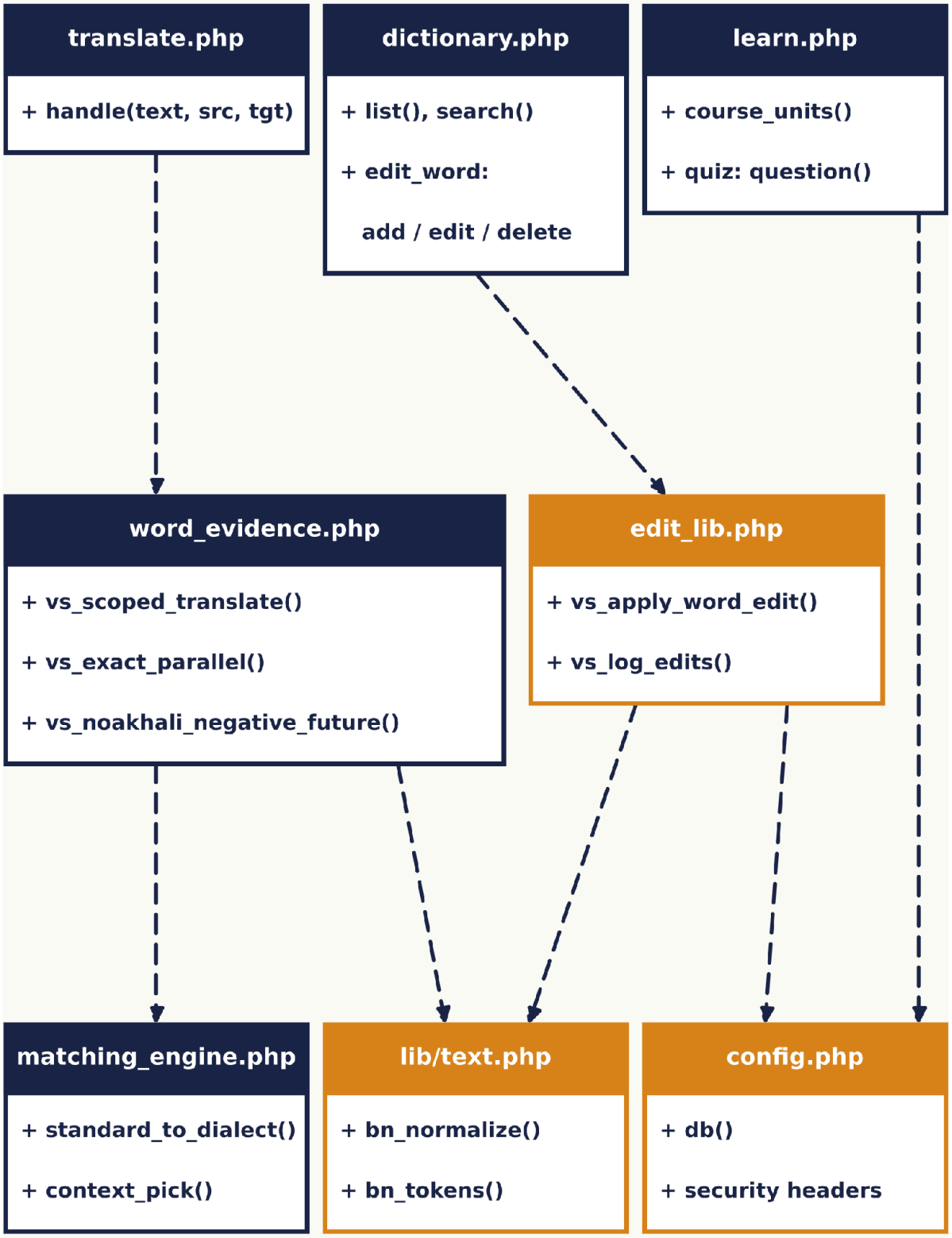


Fig. 3.5 Module / class diagram

The sequence diagram (Fig. 3.6) follows one translation request: the client sends the text, translate.php normalises it, tries an exact sentence match, falls back to the word engine if allowed, applies native-speaker rules and returns the translation with word boxes.

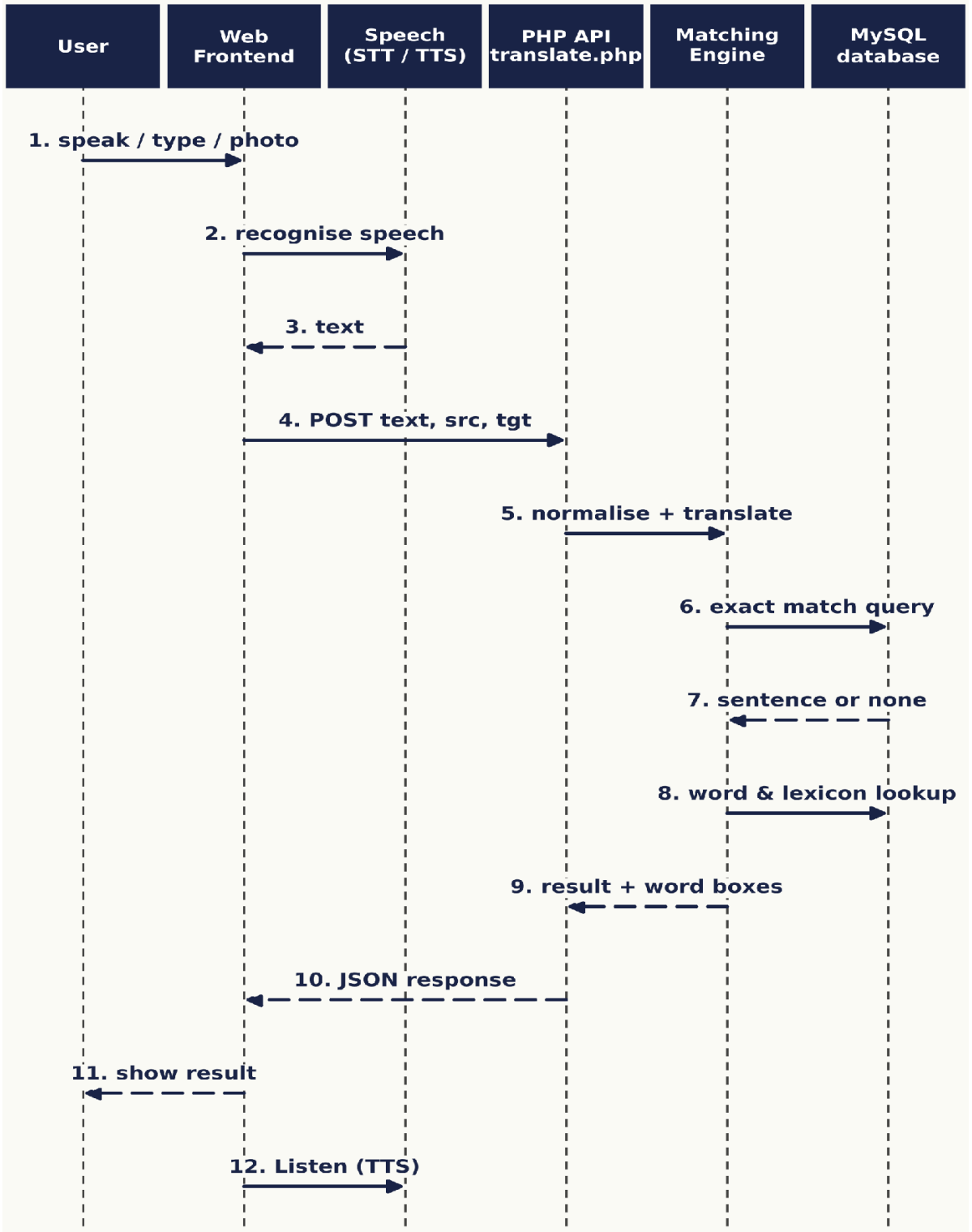


Fig. 3.6 Sequence diagram of a translation request

The activity diagram (Fig. 3.7) shows the decisions inside the translation flow.

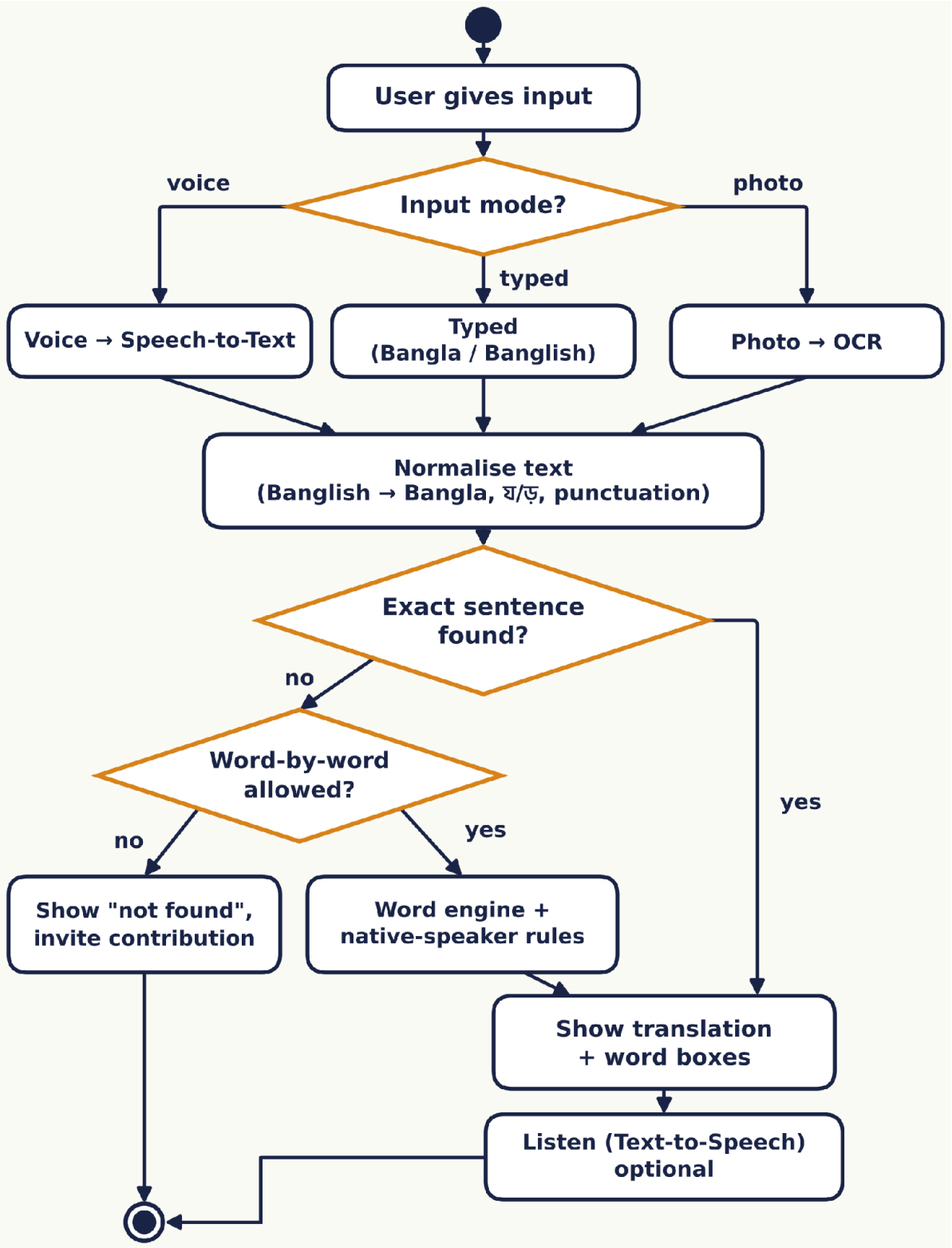


Fig. 3.7 Activity diagram of the translation process

3.5 User Interface Design

The UI was designed mobile-first, with a bottom navigation bar for the main screens. Figs. 3.8 and 3.9 show the implemented screens.

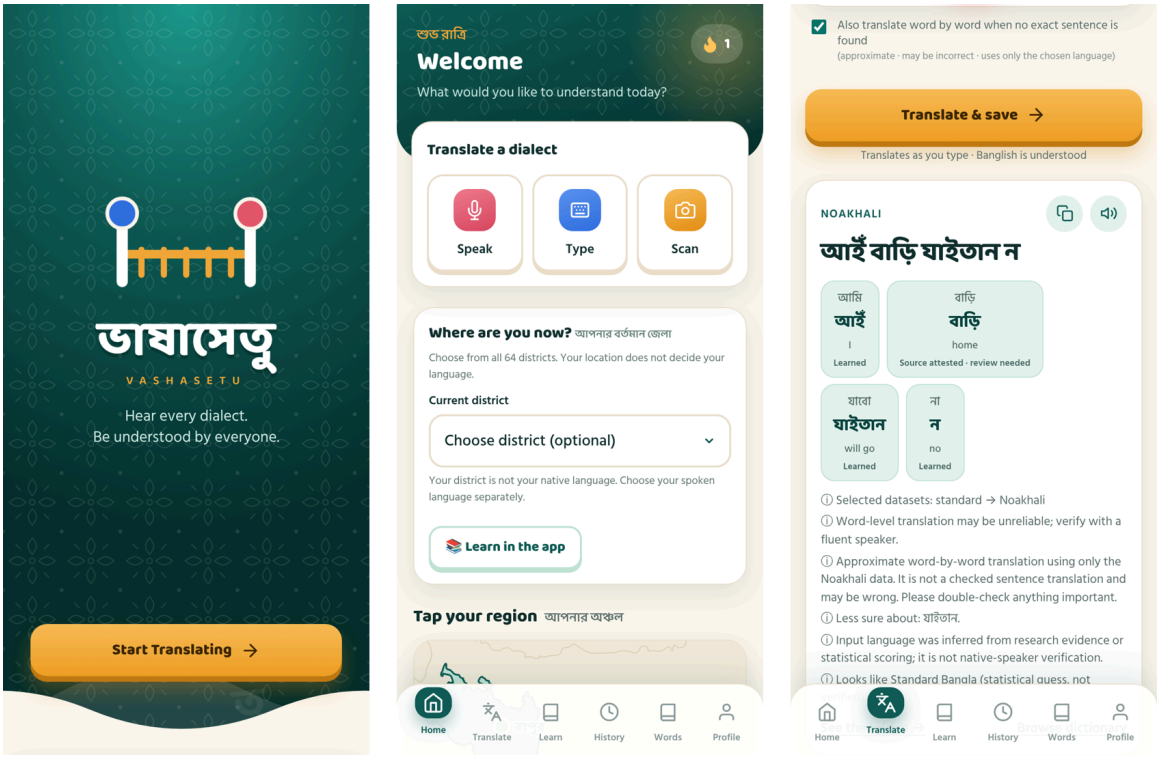


Fig. 3.8 Splash screen, Home (quick actions, district and map) and Translate result with word boxes

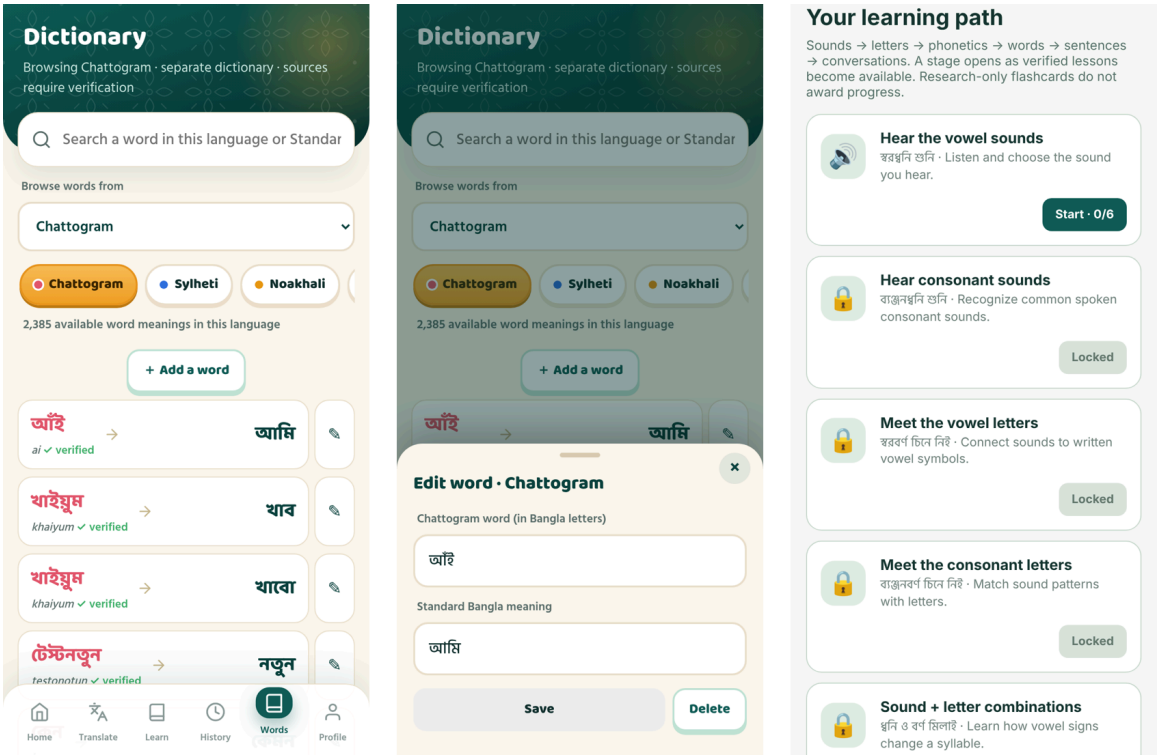


Fig. 3.9 Dictionary (verified words first), edit form, and the learning path

Design choices: the green and saffron colour scheme and alpona patterns give a Bangladeshi feel; every label is in Bangla and English; buttons are at least 44 px high; uncertain results carry a visible "needs review" label; and the install button was removed at the users' request because the browser's own "Install app" option is enough.

The learning page follows the language chosen under "I speak" (Fig. 3.10). Bangla speakers get Standard Bangla instructions; Garo learners see pictures, symbols and words from their reviewed list (for example ছি = পানি); Mro learners, whose language has no reviewed text yet, see only pictures and symbols (✓, ✗, 🔊, →), never English.

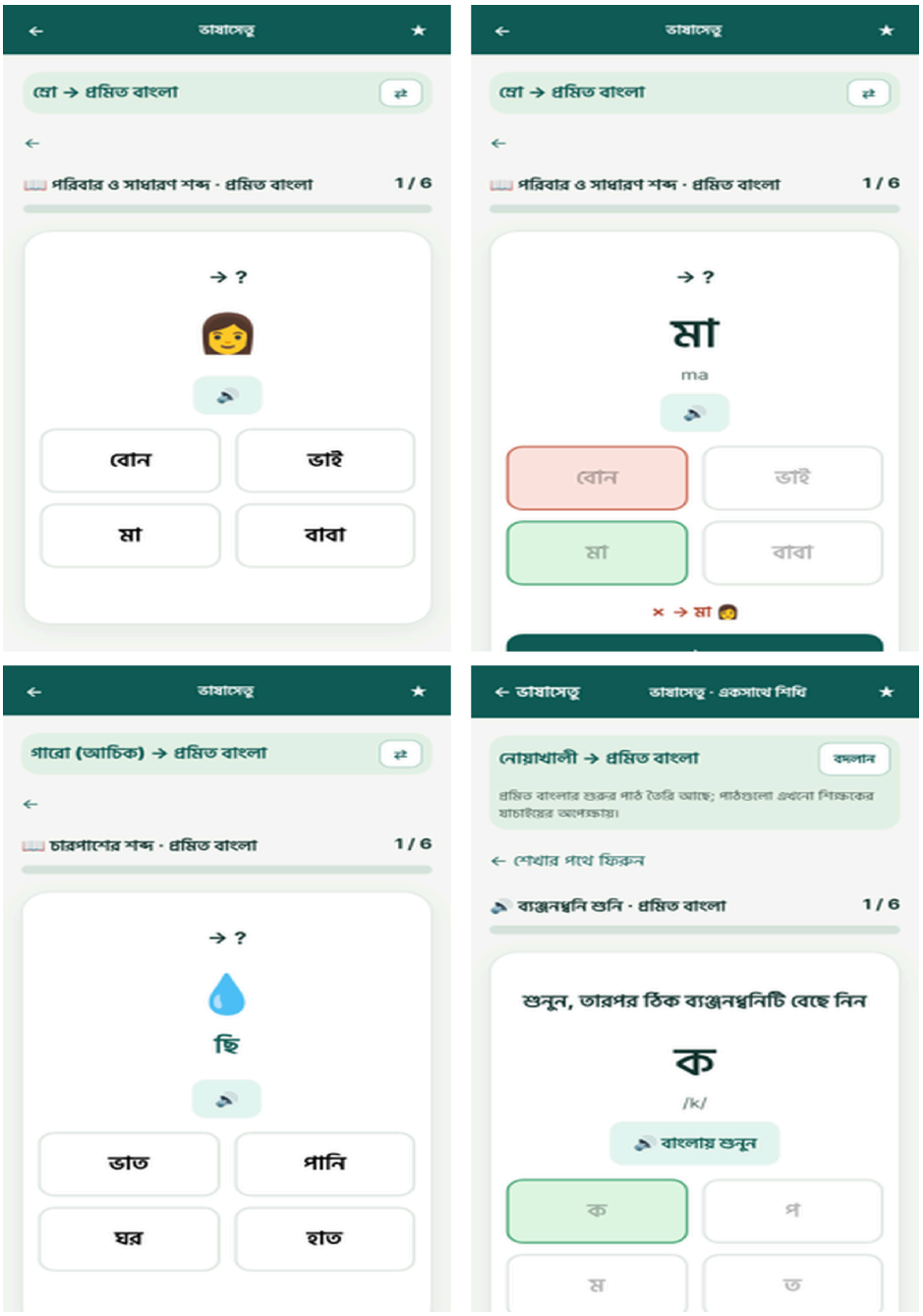


Fig. 3.10 Learning page in the learner's language. Top: a Mro learner (pictures and symbols) and the same Mro question after answering. Bottom: a Garo learner (reviewed Garo word ছি for পানি) and a Noakhali learner (Standard Bangla)

3.6 Design Decisions and Rationale

Table 3.2. Key design decisions and their rationale

Decision	Evidence / reasoning
Data-driven engine instead of a neural model	Zero budget and no GPU; every output must be explainable and correctable (Section 2.5).
Exact sentence match first, word-by-word only when the user allows it	Word-by-word output was often wrong for whole sentences; users preferred an honest "not found" with an option.
Native-speaker confirmations override corpus statistics	Speakers showed that the corpus preferred wrong forms (e.g. Noakhali আইঁ, খামু, ভালা were confirmed).
Check imported word lists against the corpora before use	A large uploaded list contained placeholders and copied rows; only confirmed entries change translations.
Normalise য়/ড়/ঢ় and punctuation	The same word was typed in two Unicode forms, so sentence matches failed (tests T01–T02).
Per-IP edit log with undo instead of mandatory login	Friends checking words found login a barrier; logging keeps edits accountable and reversible.
Script lessons before dialect lessons for indigenous learners	Chakma, Marma and Garo learners often cannot read Bangla letters.
PWA over a native app	One code base, installs from the browser, no store fees.
Learning instructions follow the learner's language; no English and no guessed indigenous text	A Mro learner reads neither English nor Bangla, and no reviewed Mro text exists; a wrong sentence would mislead learners, so pictures and symbols are used until native speakers write the text (Section 4.4).
Indigenous languages use only native-speaker reviewed words	The Garo list was reviewed by a native speaker; unreviewed data is never used, and a missing word is left unchanged with a note instead of a guess (Section 4.3).

Tools were used to refine these decisions, not to make them: for example, we decided that native words must win, and an assistant tool was then used to write the import checks and to measure how often the corpus agreed with each word (Appendix A).

4. Implementation

4.1 Introduction

The implementation follows the architecture of Section 3. This section describes the environment and repository, the core modules (data layer and translation engine), the interface and integration, and how assistant tools were used to refine the code.

4.2 Development Environment and Repository Setup

Development used XAMPP (Apache, PHP 8, MariaDB) on Windows laptops, Visual Studio Code, Chrome DevTools and an Android phone for testing. Python 3 was used for the data scripts. The repository is organised as follows:

Table 4.1. Repository structure

Path	Contents
index.php, learn.php, manifest.json, sw.js	Main pages, PWA manifest and service worker
api/	JSON endpoints: translate, dictionary, edit_word, learn, quiz, auth, history, lookup …; lib/text.php
assets/	app.js, learn.js, bdmap.js, CSS, fonts, Tesseract OCR files
database/	CSV data per language (languages/<lang>/sentences.csv, words.csv), curated.csv, lexicon.csv, schema.sql, reviewed/ lists
tools/	Python scripts: import_checked_lists.py, add_dwadash_full.py, enable_exact_matches.py …
tests/	PHPUnit suites (tests/phpunit: Unit, Data, Integration, HttpFlows – 421 tests); Python system, UI, API, security, load, reliability and accessibility suites; coverage scripts; mutation_check.py
.github/workflows/	ci.yml – GitHub Actions pipeline
setup.php, admin_edits.php	Database loader (local only) and the edit-review page (local only)
database/learn_ui.csv	Learning-page instructions: Standard Bangla, pictures/symbols, and one column per indigenous language for native speakers

Branching and commits: because the team is small, work was committed directly to the main branch. The first nine commits (19–21 October) uploaded the working local build; the following ten commits each describe one feature or fix in plain words (e.g. "Learn page in the learner's own language; dialect-to-dialect word boxes show the target word; Bridge screen fixed"). Using feature branches and pull requests is listed as an improvement in Section 6.2.

4.3 Core Module Implementation

Data layer. All data lives in CSV files so that it can be reviewed and versioned. setup.php creates the schema and loads, for each language, its sentence pairs and word list. Table 4.2 shows the size of the data. Before loading, every string passes through bn_normalize(), which removes zero-width characters, decomposes the nukta letters and strips punctuation:

```
function bn_normalize(string $s): string {
    // remove zero-width joiners
    $s = str_replace(["\u{200D}", "\u{200C}"], '', $s);
    // ড় ঢ় য় -> base letter + nukta, so both typings match
    $s = strtr($s, ["\u{09DC}" => "\u{09A1}\u{09BC}",
                    "\u{09DD}" => "\u{09A2}\u{09BC}",
                    "\u{09DF}" => "\u{09AF}\u{09BC}"]);
    // punctuation -> space, then squeeze the spaces
    $s = preg_replace('/[?!,.;:।"()\[\]—-]+/u', ' ', $s);
    return trim(preg_replace('/\s+/u', ' ', $s));
}
```

Table 4.2. Data loaded per language (sentence pairs, dictionary words, sentences enabled for exact match)

Language	Sentence pairs	Dictionary words	Exact-match sentences
Chattogram	25,345	2,384	18,102
Sylheti	21,810	3,210	14,762
Noakhali	7,899	537	4,896
Mymensingh	6,553	459	5,347
Barishal	4,845	728	3,718
Rajshahi	2,106	448	1,354
Rangpur	1,161	113	696
Narail / Kishoreganj / Narsingdi / Tangail	about 500 each	52–113	209–355
Chakma / Marma / Garo (MELD)	799 / 292 / 314	218 / 51 / 86 (Garo reviewed)	all (phrases)

Translation engine. vs_scoped_translate() first looks for an exact sentence pair in the selected language (native-speaker sentences first). A corpus pair is enabled for exact matching only when no dataset gives a different translation for either side, it has 2–25 Bangla words, its words line up with known word pairs, and it does not contradict a native-speaker word (tools/enable_exact_matches.py). If there is no exact match and the user allowed approximate translation, standard_to_dialect() scores candidate words from three sources: curated entries (weight 3.5), native/community dictionary entries (4.0) and the lexicon learned with IBM Model 1 from the corpora (125,787 word pairs). Rare candidates (seen fewer than 3 times) get a penalty and weak candidates below 0.45 are dropped; context_pick() uses word bigrams to choose between close candidates. For dialect → dialect, the text goes through Standard Bangla.

Native-speaker rule. A Noakhali speaker confirmed "আমি ভাত খাবো না = আইঁ ভাত খাইতান ন". The corpus does not contain this pattern, so we wrote a rule: a future verb followed by না becomes stem + ইতান followed by ন (যাবো না → যাইতান ন, করবো না → কইরতান ন). The rule works in both directions:

```
// Noakhali: <verb>বো না  ->  <stem>ইতান ন
if ($next !== 'না' || !preg_match('/^(.+?)বো?$/u', $w, $m)) continue;
$stem = $m[1];                        // খা, কর, দেখ ...
if (preg_match('/^.+[ি]ই$/u', $stem))      // খা -> খাইতান
    $form = preg_replace('/ই$/u', '', $stem) . 'ইতান';
// ... similar cases: কর -> কইরতান, দেখ -> দেইখতান
$rows[$i]['tgt']     = $form;
$rows[$i + 1]['tgt'] = 'ন';          // না -> ন
```

Import checks. tools/import_checked_lists.py compares every entry of a user-supplied list with the corpus and labels it Confirmed, Contradicted or Not found. Only confirmed forms used by at least a quarter of the corpus change translations; the others appear in the dictionary as "needs native review". The result of every entry is written to a check report.

Native-speaker reviewed lists. In November 2026 native speakers reviewed the whole Noakhali word list (537 pairs), the Chattogram list, the whole Barishal list (728 pairs) and the whole Garo list (86 pairs) in Google Sheets. tools/apply_native_review.py loads a reviewed sheet: an entry such as বাচ্ছা/বানো becomes a main form and an approved alternative (native_speaker_reported / native_speaker_alternative). tools/clean_word_lists.py fixes only mistakes that are certain, such as a vowel sign typed twice or ো typed in two parts; unusual spellings were sent back to the reviewer, who approved eight of them as written. For indigenous languages the translator uses only reviewed words in both directions, and bn_native_normalize() keeps the apostrophe and raised dot that are part of Garo words (চা'আ = খাওয়া, হি•গুব্বা = এটি).

4.4 Interface Implementation and System Integration

The interface is a single-page app (index.php + assets/js/app.js) with screens for Home, Translate, Learn, History, Words (dictionary) and Profile, switched by a bottom navigation bar. Each screen calls one API endpoint with fetch() and renders the JSON result. The main integration points are:

- Translate → api/translate.php: returns translated_text, detected variety, quality notes, word_meanings for the word boxes and the word pairs for the Bridge screen; from one dialect to another the boxes show the target dialect word, with the Standard Bangla word underneath.
- **Words** → api/dictionary.php (list, search, ordering: verified first, then dialect-specific forms, then forms identical to Standard) and api/edit_word.php (add, edit, delete).
- Learn → learn.php with api/learn.php (units, progress, and the instruction text in the learner's language from database/learn_ui.csv; word questions show a picture and, when reviewed, the learner's own word) and api/quiz.php (word and sentence quizzes for any language pair).
- **Voice and photo** → Web Speech API with a Stop button; Tesseract.js runs OCR in the browser and puts the text into the input box.
- **Accounts** → api/auth.php (register, login, logout) with PHP sessions.

Banglish input is converted to Bangla script before translation. The service worker caches the app shell so the app opens quickly and can be installed.

4.5 Tool-Assisted Refinement

For key modules we first decided what the module must do and how it should behave, then used an AI assistant (Claude Opus 5.5) to write or refine the code, and finally verified the result ourselves. Examples:

- **Translation engine.** Our design: exact match first, native words win, approximate only on request. The assistant wrote and refactored the PHP scoring and the data scripts. Verification: we tested sentences we know, asked native speakers, and kept the 20 API tests passing. Several outputs were rejected – e.g. Noakhali খাবো was first translated to the corpus form; the speaker corrected it to খামু and the rule was changed.
- **Dictionary ordering and editing.** Our design: verified words first, then words that look dialect-specific, and editing of both the dialect word and the meaning with an undo per address. The assistant implemented the SQL and the log; we checked the order in the Sylheti and Noakhali lists.
- **Security review.** We asked the assistant to review the code for data-security issues; we accepted fixes such as login throttling, session id regeneration and blocking setup.php from the public link, and tested each one (T17–T20).
- Testing. Our design: the test levels, the techniques (equivalence partitioning, boundary values) and the expected values, taken from the SRS and the native-speaker data. The lab named Qodo and Diffblue; Qodo needs its IDE plugin and an account and Diffblue Cover only generates Java tests, so the assistant generated 25 edge-case tests instead. We checked every expected value: 22 were correct, 2 wrong and 1 questionable, and those were corrected (Software Test Report, Section 10).

All tool use is recorded in Appendix A. The assistant was not used to make decisions about dialect words – those came from native speakers and the datasets.

5. Testing, Deployment and Result Analysis

5.1 Introduction

This section presents the testing and quality assurance, the code review and security work, documentation and version control, the CI/CD pipeline and deployment, and the results against the objectives.

5.2 Testing and Quality Assurance

Testing was planned and reported in a separate Software Test Report (v1.0). It covers every level: unit, data and integration tests with PHPUnit 9.6 [16]; system and API tests in Python; end-to-end UI tests with Playwright [17] on a phone-sized Chromium browser; performance, security, reliability and accessibility tests (axe-core, WCAG 2.1 AA [18]); manual testing of every screen on Android phones; and an accuracy evaluation on held-out corpus sentences (Section 5.6). Each test case has an input, an expected result and the actual result. On the final build 84 test cases with 673 individual checks were run and 83 cases pass (Table 5.2). Table 5.1 lists the API test cases, run again on the final build.

Table 5.1. API test cases and results (final build, 3 November 2026)

ID	Test case	Input	Expected result	Actual	Status
T01	Standard → Chattogram uses native form	আমি ভাত খাবো	আঁই ভাত খাইয়ুম	As expected	Pass
T02	Banglish input	ami vat khabo	contains খাইয়ুম	As expected	Pass
T03	Native sentence wins (Noakhali)	আমি ভাত খাবো না	আইঁ ভাত খাইতান ন	As expected	Pass
T04	Noakhali negative-future rule	আমি বাড়ি যাবো না	contains যাইতান ন	As expected	Pass
T05	Mymensingh -বাম verb	আমি ভাত খাবো	contains খাইবাম	As expected	Pass
T06	Dialect → dialect via Standard	আই ভাত খামু (Noakhali→Chattogram)	non-empty translation	As expected	Pass
T07	Dialect → Standard	তুই কই যাস (Sylheti)	non-empty translation	As expected	Pass
T08	Chakma exact sentence (MELD)	মুই গম আগং	আমি ভালো আছি	As expected	Pass
T09	Auto-detect keeps Standard	আমি ভাত খাবো (auto)	variety = standard	As expected	Pass
T10	Unsupported language rejected	source = klingon	HTTP 4xx	As expected	Pass
T11	New Dwadash dialect	আমার কাছে টাকা নেই → Rangpur	contains মোর	As expected	Pass
T12	Word box for every word	আমি ভাত খাবো না	all words known	As expected	Pass
T13	Verified words listed first	dictionary Noakhali	first row verified	As expected	Pass
T14	Indigenous dictionary has phrases	dictionary Chakma	more than 500 rows	As expected	Pass
T15	Quiz for a low-data dialect	quiz Tangail	a question	As expected	Pass

ID	Test case	Input	Expected result	Actual	Status
T16	Dictionary edit saved	add পরীক্ষাশব্দ	listed once	As expected	Pass
T17	Register and login	test account	success	As expected	Pass
T18	Login throttling	9 wrong passwords	HTTP 429 after 8	As expected	Pass
T19	Learn path for indigenous learner	Chakma → Noakhali	script lessons first	As expected	Pass
T20	setup.php blocked via tunnel	request with tunnel headers	refused	As expected	Pass

Table 5.2. Results by test level on the final build

Level (tool)	Passed / run	What was tested
Unit (PHPUnit)	202 / 202	Password rules, text normalisation, fuzzy matching, English grammar, Noakhali rule, validation
Data (PHPUnit)	106 / 106	All 25 word lists: complete rows, no typing mistakes or duplicates, valid spelling of reviewed words
Integration (PHPUnit, HTTP)	147 / 147	Translation flow, database, login, translate, dictionary and learn APIs, history, profile
System and API (Python)	97 / 97	Translation in every direction, input limits, dictionary, edit log, accounts, quiz, learning path
End-to-end UI (Playwright)	43 / 43	Every screen on a phone-sized browser, voice, scan, Bridge, install, offline start, learner-language Learn page
Security (Python, OWASP Top 10)	48 / 48	Password guessing, flooding, SQL injection, XSS, CSRF, sessions, access control, headers, fuzzing
Performance (load script)	10 / 14	20–300 users, 500-request spike, 3-minute soak; 100 users non-stop miss the 3 s target (D34)
Reliability and accessibility	16 / 16	40 people editing at once, MySQL and Apache restarts; axe-core on 10 screens, keyboard, 200% zoom
Total	**669 / 673**	**84 test cases: 83 pass; 38 defects found, 37 fixed**

Coverage: line coverage was measured with PCOV, both inside PHPUnit and for every web request made by the other suites. All suites together run 90.1% of the PHP code (2,517 of 2,794 lines; translation engine 91.1%, target 60%). Mutation testing planted 24 small bugs one at a time; the tests caught 23, and the last one cannot be seen from outside (an equivalent mutant). Voice and photo input were tested manually on Android phones because they need a microphone and a camera.

5.3 Code Review, Refactoring and Security Hardening

Code was reviewed in three passes: a functional review after each native-speaker test round, a full security review before deployment, and a test-driven review in November in which every test type was run. Findings were triaged by severity (S1 data loss or a working attack, S2 wrong result, S3 some inputs or a hardening gap, S4 code quality). Table 5.3 lists the main findings.

Table 5.3. Review findings and fixes

Finding	Severity	Fix
Dictionary page error (missing require of a library file)	Must-fix	Added the require; list now loads for every language
28 trusted entries contradicted by the corpus (e.g. Chattogram করমু)	Must-fix	Removed from curated.csv and lexicon.csv
Sentence match failed for য়/ড় typed in two Unicode forms	Must-fix	bn_normalize() decomposes nukta letters
setup.php could reset the database through the public link	Must-fix	Only runs for local requests
No limit on password guesses	Must-fix	login_attempts table: 8 tries per 15 minutes, HTTP 429
Session fixation possible	Should-fix	session_regenerate_id() on login and registration
Cookies not marked Secure behind the tunnel	Should-fix	Secure flag when HTTPS (also via X-Forwarded-Proto)
Data and tool folders downloadable	Must-fix	.htaccess deny rules for database/, tools/, docs/ and data file types
Error messages revealed database details	Should-fix	Generic messages; details only in the server log
Anyone can edit the dictionary	Accepted risk	Every edit logged with IP and old value; undo per IP in admin_edits.php
Ambiguous SQL columns after joins; quiz.php 500 error	Must-fix	Sub-query alias; valid_dialect() helper
Login limit bypassed by faking the visitor-address header (D11)	Must-fix (S1)	The header is trusted only from the tunnel on our own computer
40 people editing at once wiped the word list from 81 rows to 5 (D21)	Must-fix (S1)	File lock and atomic write for every shared file
Password length counted bytes, so Bangla passwords were judged wrongly	Must-fix	mb_strlen() counts letters
Bridge screen empty; dialect-to-dialect boxes showed the Standard word; Learn page in English for indigenous learners (D35–D37)	Must-fix	Helper restored; target word in every box; instruction table in the learner's language
Garo words with an apostrophe split in two; Standard → Garo never used the reviewed list	Must-fix	bn_native_normalize(); reviewed Garo words used in both directions

Refactoring: shared editing logic was moved to api/edit_lib.php, text handling to api/lib/text.php, and the data scripts were made re-runnable in a fixed order. In total 38 defects were found (D1–D38); 37 are fixed and one is open (D34, speed with 100 users at once). Software licences were respected: all datasets are CC BY 4.0 or MIT and are credited in DATA_SOURCES.txt; the fonts (SIL OFL) and Tesseract.js (Apache 2.0) licences are included in the repository. Passwords are stored only as bcrypt hashes, all queries use prepared statements, and composer.json, phpunit.xml and vendor/ are blocked from the web. The remaining risk is that MySQL root has no password on the development laptop; this must be set before real public use.

5.4 Documentation and Version-Control Management

Documentation produced: the project proposal and system model (CSE 355), the SRS (Section 2.4), README.md (installation, phone use, features, how edits work, data quality, folder guide, troubleshooting, test results, security), DATA_SOURCES.txt (datasets and licences), the Software Test Report v1.0 (test plan, 84 test cases, defect log), the instruction sheet for native speakers (database/learn_ui.csv), a change log in docs/, the presentation, and this report.

Version control: the GitHub repository https://github.com/Muskan37/Vashasetu holds the code, data and tests. The history has 19 commits from 19 October to 3 November 2026. Commit messages describe the change in plain words. Pull requests were not used because both members worked on one shared copy; this is a weakness we plan to fix (Section 6.2).

5.5 CI/CD, Deployment and Release Management

The CI pipeline (.github/workflows/ci.yml, Fig. 5.1) runs on GitHub Actions [3] for every push and pull request: it starts a MySQL 8 service, sets up PHP 8.2 with PCOV, Node 20 and Python, checks the syntax of every PHP, JavaScript and Python file, loads the database with setup.php, starts the PHP server, runs PHPUnit with coverage and runs the API tests. A failing step blocks the change from being considered ready. Before pushing, the same steps were run on the development machine; every step passed (Table 5.4).

Table 5.4. Result of the pipeline steps (local run before each push)

Step	Result	Time
Lint PHP (23 files)	Pass	< 1 s
Check JavaScript syntax	Pass	1 s
Compile Python scripts	Pass	< 1 s
Load the database (73,133 sentence pairs, 8,553 word meanings)	Pass	13 s
Start PHP server	Pass	2 s
Run 20 API tests	Pass (20/20)	2 s
Run PHPUnit with coverage (421 tests, 43,985 assertions)	Pass (421/421)	–

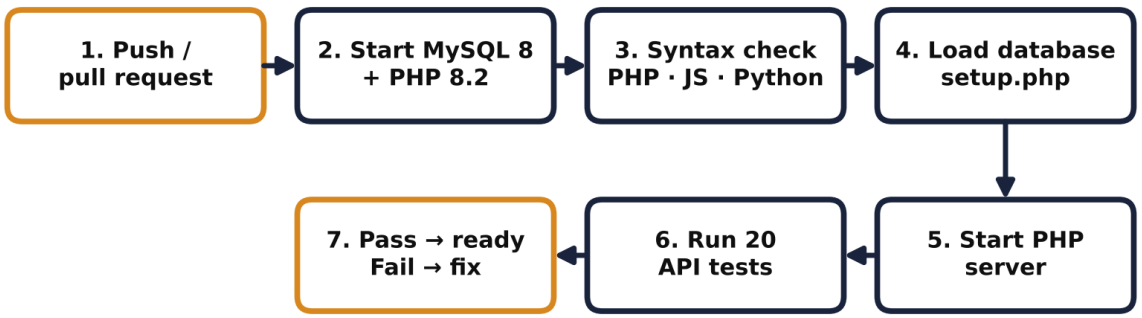


Fig. 5.1 CI pipeline stages

Deployment: the app runs on XAMPP on a laptop and is published over HTTPS with a free Cloudflare quick tunnel, so it can be opened and installed on any phone. Because the quick-tunnel address changes on every restart, the live link is given at the demonstration. Releases: v1 (first local build, uploaded 19–21 Oct), v2 (MELD data, dictionary editing, 22 Oct), v3 (checked word lists and native rules, 22–23 Oct), v4 (all Dwadash dialects, Bangla-script lessons, security fixes, per-IP edit log, 25 Oct), v5 (native-speaker reviewed word lists, PHPUnit suite with coverage and mutation testing, fixes D25–D33, 2 Nov), v6 (Learn page in the learner's language, dialect-to-dialect word boxes, Bridge fix, 3 Nov).

Repository: *https://github.com/Muskan37/Vashasetu*

5.6 Results and Discussion

Functional coverage: all eight functional requirements of the SRS are implemented and tested (Tables 5.1 and 5.2; the requirement traceability matrix is in the Software Test Report). Objectives O1–O5 and O7 are met; O6 is met for five of the six main dialects (Table 5.5).

Accuracy was measured on 150 held-out corpus sentences per dialect (Rajshahi 64), counting a word as correct only if it matches the reference spelling exactly:

Table 5.5. Word-level accuracy on held-out corpus sentences

Dialect	Standard → dialect	Dialect → Standard
Barishal	79%	87%
Chattogram	55%	76%
Mymensingh	79%	90%
Noakhali	66%	85%
Rajshahi	89% (small set)	93%
Sylheti	61%	80%

Dialect → Standard is easier than Standard → dialect because one Standard word has many dialect spellings (আঁই / অ্যাঁই, যাইয়ুম / যাইয়্যুম); many "errors" are such spelling variants, so real accuracy is higher. Chattogram and Sylheti are hardest because their verb forms differ most from Standard Bangla. These numbers are not a native-speaker evaluation.

Performance: a single translation takes a median of 7 ms on the laptop server, well inside the 1 s requirement, and the first screen appears after 1.5 s even on slow 4G. Under stress, with 100 users sending requests non-stop on 2 CPU cores, 95% of the replies arrive within 3.85 s and none fail; this misses our own 3 s target and is logged as defect D34. Usability feedback from friends and classmates led to concrete changes: a Stop button for voice, editing both the dialect word and the meaning, removing the install button, putting verified words first in the dictionary, and fixing the bottom navigation on small phones. Native speakers reviewed the Noakhali, Chattogram, Barishal and Garo word lists, and their corrections now override the corpus. Using the app on a phone in November showed three more problems – the Bridge screen was empty, dialect-to-dialect boxes showed the Standard word, and the Learn page was in English for Mro and Garo learners – which were fixed and are now covered by tests.

Mid-term feedback. At the mid-term demonstration the instructor pointed out that the database contained wrong dialect entries and asked us to fix it, and to make sure that the Scan (photo) and voice inputs work. In response we (i) checked every trusted entry against the corpora and removed 28 that the corpus contradicts, (ii) made every imported word list pass the corpus check before it can change a translation, and let native speakers' confirmations override the corpus (Sections 4.3 and 5.3); and (iii) fixed voice input, which was not writing any text, added a Stop button, and re-tested voice and Scan on a laptop browser and an Android phone.

6. Conclusion and Future Work

6.1 Conclusion

Vashasetu delivers what Section 1 set out to build: a free, installable web app that translates between Standard Bangla, English, eleven regional dialects and Garo (and between dialects), explains every word, offers an editable dictionary with an accountable edit log, and teaches with a learning path that starts from the Bangla script and speaks to learners in their own language. It combines about 73,000 sentence pairs from nine research datasets with word lists reviewed by native speakers, and it labels honestly what has not been checked. In the final test 83 of 84 test cases pass (673 checks), the tests cover 90.1% of the PHP code, all security tests pass, and word-level accuracy from dialect to Standard Bangla is 76–93% on held-out sentences.

6.2 Future Work

- Ask native speakers of the remaining dialects to review the most frequent 1,000 words, and finish the indigenous datasheets as well.
- Add audio recordings of native speakers for Listen and for the learning lessons.
- Try a small neural model for sentence-level fluency, keeping the word boxes and native rules as a check.
- Require login for editing once the community grows, with reviewer roles; set a MySQL password and automatic backups.
- Move to permanent hosting with a fixed domain, and use feature branches, pull requests and code review on GitHub.
- Fix the speed limit with many users at once (D34) with more Apache workers or a cache for repeated translations.
- Extend the indigenous-language support beyond MELD phrases with community partners, starting with Mro, Garo and Chakma learning instructions in database/learn_ui.csv.

References

[1] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill, 2020.

[2] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2015.

[3] GitHub, "GitHub Actions documentation." docs.github.com. https://docs.github.com/actions (accessed Oct. 26, 2026).

[4] "Vashantor: Bangla regional dialect parallel corpus," Mendeley Data, 2024, doi: 10.17632/bj5jgk878b.2.

[5] "ONUBAD: Bangla regional dialect to Standard Bangla parallel corpus," Mendeley Data, 2024, doi: 10.17632/6ft99kf89b.2.

[6] "Kothon: Large-scale dataset for machine translation of Chittagonian and Sylheti into Standard Bangla," Mendeley Data. https://data.mendeley.com/datasets/2fv6vf9v2z (accessed Oct. 26, 2026).

[7] "Dwadash: Multi-dialectal translation across twelve Bengali variations," Mendeley Data. https://data.mendeley.com/datasets/v9cf66fk2t (accessed Oct. 26, 2026).

[8] "ChatgaiyyaAlap: Chittagonian to Standard Bangla," Mendeley Data. https://data.mendeley.com/datasets/wtms9xbkkw (accessed Oct. 26, 2026).

[9] "BD-Dialect: A multiregional Bangla language dataset," Mendeley Data, v2, 2026, doi: 10.17632/k769s4vk5z.2.

[10] "MELD: Chakma, Marma and Garo phrase dataset," Mendeley Data, v4, 2025, doi: 10.17632/dy5dyfygbp.4.

[11] P. F. Brown, S. A. Della Pietra, V. J. Della Pietra, and R. L. Mercer, "The mathematics of statistical machine translation: Parameter estimation," Computational Linguistics, vol. 19, no. 2, pp. 263–311, 1993.

[12] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.

[13] OWASP Foundation, "OWASP Top 10: 2021." owasp.org. https://owasp.org/Top10/ (accessed Oct. 26, 2026).

[14] MDN Web Docs, "Progressive web apps." developer.mozilla.org. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (accessed Oct. 26, 2026).

[15] Anthropic, "Claude Opus 5.5." claude.ai. https://claude.ai (accessed Oct. 26, 2026).

[16] S. Bergmann, "PHPUnit – The PHP testing framework." phpunit.de. https://phpunit.de (accessed Nov. 9, 2026).

[17] Microsoft, "Playwright." playwright.dev. https://playwright.dev (accessed Nov. 9, 2026).

[18] Deque Systems, "axe-core: accessibility engine for automated web UI testing." github.com. https://github.com/dequelabs/axe-core (accessed Nov. 9, 2026).

Appendix A: Tool Usage Log

Every use of an assistant tool is recorded below. "Own design first" states what we had decided or designed before using the tool.

Table A.1. Tool usage log

Wk	Task / artifact	Tool used	Own design first (what we did)	Tool's role (refine / explore / verify)	How the output was verified	Decision (accepted / modified / rejected)
1–3	Proposal, SRS, system models (CSE 355)	draw.io, Figma	All content written by us	Drawing only	Reviewed by the CSE 355 instructor	Accepted
4–6	First local build (V1–V6)	AI coding assistant	Screens, modules and WBS from the proposal	Generated first code	Ran on XAMPP; manual tests	Modified
7–9	Translation engine, data import (MELD, Dwadash, BD-Dialect)	Claude Opus 5.5	Rules: exact match first, native words win, approximate only on request	Wrote and refactored PHP and Python scripts	Known sentences, native speakers, API tests	Modified (several outputs rejected)
9	Checking uploaded word lists	Claude Opus 5.5	We decided lists must be checked against the corpus before use	Wrote the check script and report	Read the check report; native confirmation of আইঁ, হুত, ঝী, ভালা, খামু	Accepted
10	Noakhali negative-future rule	Claude Opus 5.5	Native speaker gave the sentence আইঁ ভাত খাইতান ন	Generalised it into a rule	Tests T03–T04; speaker checked examples	Accepted
10	Dictionary editing, ordering, per-IP log	Claude Opus 5.5	We specified the behaviour (edit both fields, verified first, undo per IP)	Implemented SQL, API and page	Manual checks, T13, T16	Accepted
11	Security review	Claude Opus 5.5	We asked for a data-security review	Found issues, proposed fixes	Tests T17–T20; manual tunnel test	Accepted
12	Tests, CI workflow, diagrams, report draft	Claude Opus 5.5	Test cases chosen from our SRS; report content from our work	Wrote test code, workflow, diagram sources, draft text	Ran tests (20/20); read and corrected the report	Modified (cover, team roles and diagrams changed after our review)

Wk	Task / artifact	Tool used	Own design first (what we did)	Tool's role (refine / explore / verify)	How the output was verified	Decision (accepted / modified / rejected)
13	Native-speaker review import and word-list cleanup	Claude Opus 5.5	Native speakers reviewed the Noakhali, Chattogram, Barishal and Garo sheets; spelling questions were decided with them	Wrote the import and cleanup scripts	Compared counts with the sheets; data tests (106)	Accepted (8 spellings approved as written after asking the reviewer)
13	Full testing campaign and Software Test Report	Claude Opus 5.5; PHPUnit, Playwright, axe-core, PCOV	Test plan, levels and expected values from our SRS and native-speaker data	Generated test code and 25 edge-case tests; ran coverage and mutation testing	Checked every expected value: 22 correct, 2 wrong, 1 questionable – corrected; reran all suites	Modified
14	Learn page in the learner's language; Garo translation; Bridge fix	Claude Opus 5.5	Rule: no English and no guessed indigenous text; only reviewed Garo words	Implemented the instruction table, symbols and fixes	Manual tests on phones; tests TC-35–TC-37 and UI-41–UI-43	Accepted
14	Presentation slides	Claude Opus 5.5	Content and structure from this report and the department template	Slide layout and visuals	Reviewed and corrected by us	Modified

Appendix B: Team Contribution Record

Each member's contribution is shown below (see also Table 1.1). The repository history is summarised below.

Repository: https://github.com/Muskan37/Vashasetu

Member (name, ID)	Modules / tasks owned	Commits / pull requests	Documentation, testing and review role
Shobnom Sultana Muskan, 0432320005101142	Back end and front end: translation engine, dictionary and editing, voice (speech, Listen, OCR), all screens, navigation and API integration, data import and checking, security, deployment	All 19 commits (19 Oct – 9 Nov 2026) were pushed from the Muskan37 account	README, DATA_SOURCES, automated test suites, Software Test Report, CI workflow, report
Sadia Akter Liza, 0432320005101110	Shared work packages 1.0 Requirements and 2.0 Design: co-authored the CSE 355 project proposal and system model (component, activity, sequence and fishbone diagrams); manual testing of the app on phones	No commits in the repository; work shared through the proposal and system-model documents	Project proposal and system-model documents (CSE 355); manual testing on Android phones

Justification Report

Below is the CO-PO Mapping and Complex Engineering Problem (CEP) Justification Report for the project titled "Vashasetu (ভাষাসেতু): A Regional Bangla Dialect Translator and Learning App", developed for the course CSE 416 Software Project Design and Development (Autumn 2026).

CO-PO Mapping

CO No.	Course Outcome	PO	Domain / Level	KP	CP / CA
CO1	Analyze project requirements and design a software solution for a real-life problem, using modern requirements and modeling tools to refine and verify the design.	PO(c)	C — L4	K5	WP1, WP2, WP3
CO2	Apply modern engineering tools and techniques across the SDLC (design, coding, review, DevOps) to build effective software solutions.	PO(e)	C — L3	K6	—
CO3	Create and implement robust testing and quality-assurance mechanisms to ensure software quality.	PO(i)	C — L5	—	WP1, WP2, WP3
CO4	Implement, deploy and manage a real-life complex software product end-to-end, with version control, CI/CD and release management.	PO(k)	C — L6	—	EA1, EA2, EA3
CO5	Investigate a real-life problem by evaluating alternative solutions and justifying design decisions with evidence.	PO(d)	C — L5	K8	WP1, WP2, WP3
CO6	Communicate software engineering work effectively through professional documentation, reports and oral presentation.	PO(g)	C/A — L4 / A3	K7	EA1, EA2, EA5
CO7	Function effectively as a member of a two-person team, sharing responsibility and integrating individual contributions.	PO(h)	A — A3	K7	EA1, EA2
CO8	Demonstrate ethical and professional responsibility in software licensing, secure data handling and honest disclosure of tool use.	PO(f)	A — A3	K7	—

Domain: C = Cognitive (Bloom L1–L6), P = Psychomotor (P1–P7), A = Affective (A1–A5). KP = Knowledge Profile (K1–K8). CP = Complex Problem attributes (WP1–WP7). CA = Complex Activity attributes (EA1–EA5).

Mapping of COs with Program Outcomes (POs)

CO	(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)
CO1			**3**									
CO2					**3**							
CO3									**3**			
CO4											**3**	
CO5				**2**								
CO6							**2**					
CO7								**2**				
CO8						**2**						

Correlation strength: 3 = High, 2 = Medium, 1 = Low.

Justification of PO through COs

CO	PO	Justification	Where in this report
CO1	**PO(c)**	Dialect communication problems were analysed with users and native speakers and turned into an SRS (FR1–FR8, NFR1–NFR4) and design models: use-case, ER, module, sequence and activity diagrams and the architecture. Tools (Graphviz, draw.io) were used to draw and check the models.	Sec. 2 (2.3–2.4); Sec. 3 (3.3–3.5)
CO2	**PO(e)**	Modern tools across the SDLC: VS Code, XAMPP (PHP 8, MySQL), Git/GitHub, GitHub Actions, Python data scripts, PHPUnit with PCOV, Playwright, axe-core, Tesseract.js, Web Speech API and an AI assistant (Claude). Our own design (proposal, SRS, rules, test plan) came first; tools refined and verified it (Appendix A).	Sec. 3 (3.3); Sec. 4 (4.2–4.5); Sec. 5 (5.3, 5.5); App. A
CO3	**PO(i)**	Test strategy at every level: unit, data and integration tests (PHPUnit, 421 tests), system, API, end-to-end UI, performance, security, reliability and accessibility suites, and manual phone tests. 84 test cases, 83 pass (Tables 5.1–5.2); 90.1% line coverage; mutation testing caught 23 of 24 planted bugs; an accuracy evaluation on held-out sentences (Table 5.5). Reviews found and fixed 37 of 38 defects (Table 5.3).	Sec. 5 (5.2–5.3)
CO4	**PO(k)**	The product was built, versioned in GitHub (19 commits), integrated through a CI pipeline (lint, database load, PHPUnit with coverage, API tests), deployed over HTTPS with a Cloudflare tunnel as an installable PWA, and released in six versions. Repository: https://github.com/Muskan37/Vashasetu.	Sec. 4 (4.2–4.4); Sec. 5 (5.4–5.5); App. B

CO	PO	Justification	Where in this report
CO5	**PO(d)**	Alternatives were evaluated: Google Translate and research models (Table 2.1); neural MT vs native app vs data-driven PWA (Section 2.5). Key decisions (Table 3.2) are justified with evidence such as native-speaker corrections and test failures.	Sec. 2 (2.2, 2.5); Sec. 3 (3.6)
CO6	**PO(g)**	Professional documentation: proposal, SRS, design models, README / user guide, DATA_SOURCES, the Software Test Report, this report, and the oral presentation with a live demo.	Sec. 5 (5.4); this report; presentation
CO7	**PO(h)**	Work was shared by the WBS (Table 1.1): Muskan built the back end and the front end and wrote the automated tests; Liza did the manual testing on phones; requirements and design were shared; contributions were integrated in one repository (Appendix B).	Sec. 1 (1.5); App. B
CO8	**PO(f)**	Dataset licences (CC BY 4.0, MIT) and font and library licences are credited; passwords are hashed, queries prepared, data folders protected and edits logged; unverified meanings are labelled; all tool use is disclosed in Appendix A.	Sec. 5 (5.3); Sec. 6 (6.2); App. A

Complex Engineering Problem Justification

1. Mandatory Attribute

Attribute	Linked COs	Justification
WP1: Depth of Knowledge Required	CO1, CO3, CO5	The problem needs in-depth knowledge of Bangla phonology and script (Unicode normalisation of য়/ড়/ঢ়), statistical word alignment (IBM Model 1) from the research literature (K8), and engineering design of a layered web system (K5). See Sec. 2.2, 3.3, 4.3.

2. Supporting Attributes

Attribute	Linked COs	Justification
WP2: Range of Conflicting Requirements	CO1, CO3, CO5	Usability vs security: open editing without login vs protecting data – resolved with a per-IP edit log and undo (Sec. 3.6, 5.3). Coverage vs accuracy: translate everything vs only what is reliable – resolved with exact match first and approximate only on request, with labels (Sec. 3.6). Cost vs performance: zero budget vs fluent translation – data-driven engine instead of a neural model (Sec. 2.5).
WP3: Depth of Analysis Required	CO1, CO3, CO5	Comparison of alternatives (Sec. 2.5), corpus agreement analysis of 28 wrong trusted entries and of imported lists (Sec. 4.3, 5.3), accuracy analysis per dialect and direction (Table 5.5), coverage and mutation analysis of the tests (Sec. 5.2), load analysis (D34) and a security risk review (Table 5.3).

Range of Complex Engineering Activities (CEA) Justification

Attribute	Linked COs	Justification
EA1: Range of Resources	CO4, CO6, CO7	People: two developers, native speakers of Noakhali, Chattogram, Barishal and Garo who reviewed the word lists, classmates as testers. Data: nine research datasets (about 73,000 sentence pairs). Tools: XAMPP, Git/GitHub, GitHub Actions, Python, PHPUnit, Playwright, axe-core, Tesseract.js, AI assistant. Hosting: Cloudflare tunnel. Time: one semester (Sec. 3.2–3.3, 4.2).
EA2: Level of Interaction	CO4, CO6, CO7	Native-speaker knowledge conflicted with corpus statistics; user requests (no login, no install button) interacted with security and platform limits (browser speech and voice support, changing tunnel link). See Sec. 3.6, 5.3, 5.6.
EA3: Innovation	CO4	A translator that explains every word and its reliability; dialect-to-dialect translation through Standard Bangla; native-speaker rules and reviewed lists that override statistics; a script-first learning path whose instructions follow the learner's own language, using pictures and symbols where no reviewed text exists (Sec. 3.5, 3.6, 4.3).
EA5: Familiarity	CO6	Beyond routine CRUD work: corpus-based word alignment, data quality checks, Unicode-level text normalisation and security hardening based on OWASP principles [13] (Sec. 4.3, 5.3).

Mapping to Communication (CO6):

- **Written Technical Documents: this report, the project proposal and system model, the SRS, the README / user guide and DATA_SOURCES (Sec. 2.4, 5.4).**
- **Oral Presentations: presentation and viva with a live demonstration on a phone (translation, word boxes, dictionary editing, learning path), explaining design decisions and tool usage.**
- **Visual Representation: architecture, use-case, ER, module, sequence, activity and CI diagrams, Gantt chart, screenshots and result tables (Figs. 1.1, 3.1–3.9, 5.1; Tables 2.1–5.3).**